Python – Advanced Activity Handouts (Project & Problem-Solving Focus)

**Activity A1 – From Idea to Project Specification**

Title: Define Your Mini-Project Clearly

Goal
Turn a rough idea into a precise, buildable project description.

Instructions
Pick one idea (examples: simple hospital appointment system, dataset analyzer, quiz app, log analyzer, file organizer).

**Part A – One-page project brief**

1. Project title:
Hospital Appointment Booking System.
2. Problem you want to solve (2–3 short sentences):
In many hospitals, appointments are written manually this cause confusion, double booking and loss of records
3. Target users (who will actually use this?):
Patients, Receptionist, Doctors.
4. Inputs (what data / actions will the program receive?):
   - Patient details (name, age, phone)
   - Doctor selection and appointment date
5. Outputs (what will the user see / get?):
   - Appointment confirmation
   - List of booked appointments
6. Constraints or assumptions (keep it realistic):
   - Only one appointment per patient per day
   - System is used by one hospital only

**Part B – Core features vs "nice-to-have"**

1. List minimum features needed for a working version (MVP):
   - Add patient
   - Book appointment
   - View appointment list.
2. List optional features (only if time allows):
   - Cancel appointment
   - SMS notification
3. Final question:
   - If you had only 1 week, which ONE feature would you absolutely keep? Why?

I will keep Book Appointment feature because it is the main purpose of the system.

## Activity A2 – System Decomposition (Modules & Responsibilities)

Title: Break Project into Modules

Goal
Learn to split a project into logical modules and functions.

Instructions
Use your project from Activity A1.

**Part A – High-level modules**

1. List 3–6 modules (e.g., data_input, processing, reporting, ui):
   - Module 1: patient_management
   - Module 2: doctor_management
   - Module 3: appointment_management
   - Module 4: main_menu
2. For each module, write its main responsibility in one sentence:
   - Module 1 does: Add and store patient details.
   - Module 2 does: Store doctor details
   - Module 3 does: Book and view appointments
   - Module 4 does: Show options to user

**Part B – Functions inside modules**

For one key module (choose the most important):

1. Module name: appointment_management
2. Functions planned:
   - Function 1 name: book_appointment ()
     - Input(s): patient_id, doctor_id, date
     - Output: Confirmation message
   - Function 2 name: view_appointments ( )
     - Input(s): none
     - Output: list of appointment
   - Function 3 name: cancel_appointment ( )
     - Input(s): appointment_id
     - Output: Cancellation message
3. Draw or briefly describe how data flows between these functions:

User - book_appointment( ) → store in appointment list
→ view_appointment( )

Activity A3 – Designing Data Structures

Title: Choosing the Right Data Structures

Goal

Practice selecting lists, dicts, sets, tuples, or custom classes.

Instructions

Consider your project's core data (e.g., patient records, quiz questions, log entries, transactions).

**Part A – Describe one key entity**

1. Entity name (e.g., "Appointment", "Record", "Transaction"):

   Appointment

2. For this entity, list required fields:

appointment_id →
- Field 1: int (type: int/str/float/...)
- Field 2: patient_name (type: str)
- Field 3: doctor_name (type: str)
- Field 4: date (type: str)

**Part B – Pick implementation**

Choose one representation and justify:

1. Option A – Dictionary:

python

```
item = {
  "field1": _____,
  "field2": _____,
  ...
}
```

2. Option B – Class:

python

```
class Item:
  def __init__(self, field1, field2, ...):
    self.field1 = field1
    self.field2 = field2
    ...
```

3. Which option will you use, and why?

   I will use class because it is more organized and better for large projects

4. Write a short example of creating and using this structure:

python

```
# Example creation and simple usage
a1 = Appointment (1, "Ravi", "Dr. Sharma", "16-02-2026")
print (a1. patient_name)
```

Class

```
class Appointment:
    def __init__(self, appointment_id,
patient_name, doctor_name, date):
        self. appointment_id =
appointment_id
        self. patient_name = patient_name
        self. doctor_name = doctor_name
        self. date = date
```

**Activity A4 – Planning Tests Before Coding**

Title: Think in Test Cases (TDD Lite)

Goal
Build habit of designing tests before writing code.

Instructions
Choose one non-trivial function from your project (e.g., fee calculation, filter logic, scoring rule).

1. Function name: chuk - doctor - availability
2. In simple words, what should this function do?
   To chuk if doctor is free on a given date
3. List at least 5 test cases (inputs and expected outputs):

| Test # | Input(s) | Expected output |
|---|---|---|
| 1 1 | Doctor free | True |
| 2 2 | Doctor busy | False |
| 3 3 | Wrong doctor name | Error |
| 4 4 | Empty date | Error |
| 5 5 | Past date | False |

4. Now write a small test harness:

python

```
def my_func(...):
  # TODO: implement
  pass

# Tests
print("Test 1:", my_func(...), "expected:", ...)
print("Test 2:", my_func(...), "expected:", ...)
```

```
def chuk - doctor - availability (is-booked):
    return not is - booked.
print ("Test 1 :"
chuk - doctor - availability (False),
"expected :", True)
print ("Test 2 :",
chuk - doctor - availability (True),
"expected :", False)
```

5. Reflection:
   - Did writing tests first change how you thought about the function? How?

Writing test cases helped me think about different situations before coding

**Activity A5 – Error Handling & Robustness in a Project**

Title: Make Your Project Hard to Break

Goal

Identify and protect weak points with validation and exception handling.

Instructions

Look at your project idea and identify three likely failure points (e.g., wrong file path, invalid input, network issue).

1. Failure Point 1: Wrong file path
   - How to detect it? file not found
   - How to handle it politely for the user? Show message
2. Failure Point 2: Empty input
   - Detect: input == " "
   - Handle: Ask again
3. Failure Point 3: Invalid time
   - Detect: Wrong format
   - Handle: Show error

**Part B – Add try/except around one risky area**

Example skeleton:

python

```
file_path = input("Enter file path: ")


try:
    with open(file_path, "r") as f:
        data = f.read()
    print("File loaded successfully.")
except FileNotFoundError:
    print("File not found. Please check the path.")
except PermissionError:
    print("Permission denied. Try a different file.")
```

Adapt this pattern to your project and paste your modified code below:

python

```
# Your robust code:
file = input ("Enter file : ")

try :
    f = open (file , "r")
    print ("opened")
except fileNotFoundError :
    Print ("File not found")
```

**Activity A6 – Refactoring for Clarity**

Title: Improve Existing Code (Same Behaviour, Better Design)

Goal

Practice cleaning up messy, working code into clear, modular code.

Instructions

Take one working script (from your own work or given by teacher).
It should be correct but long, repetitive, or hard to read.

1. Paste or summarize original code name / purpose:

Hospital appointment program used to store and display patient details

2. Identify at least three problems (long function, repetition, confusing names, etc.):
   - Problem 1: long code written in one block
   - Problem 2: Repetition of printing logic
   - Problem 3: Confusing variable names
3. Plan refactoring steps (high level):
   - Step 1: Divide code into functions
   - Step 2: Use meaningful names
   - Step 3: Remove repetition.
4. After refactoring, write or paste your improved version here:

python

*# Refactored code:*

5. Reflection:
   - What became easier to understand after refactoring?

The code is now easier to read and understand. Functions reduced repetition and improved clarity

4)
```
def add_appointment (data, name, doctor, time):
    data.append ({"name": name, "doctor": doctor, "time":
                 time})

def show_appointments (data):
    for i in data:
        print (i["name"], i["doctor"],
i["time"])

appointments = []
add_appointment (appointments, "Ravi", "Dr. Sharma", "10 AM")
add_appointment (appointments, "Anita", "Dr. Kumar", "11AM")
show_appointments (appointments.
```

Activity A7 – Building a Command-Line Interface (CLI)

Title: Make Your Project Usable from Terminal

Goal

Learn to take arguments and options from the command line.

Instructions

Turn some functionality of your project into a small CLI tool.

1. Define a simple CLI command:
   Example:
   - python student_tool.py add --name Alice --score 85
2. What should the tool do in ONE sentence?

The tool allows the user to add and view hospital appointments using command line.

3. Sketch CLI arguments:

| Argument / Option | Meaning |
|---|---|
| name | Patient name |
| doctor | doctor name |
| time | Appointment time |

4. Implement with argparse (skeleton):

python

```
import argparse

parser = argparse.ArgumentParser(description="Describe your tool here")
parser.add_argument("--name", type=str, help="Student name")
parser.add_argument("--score", type=int, help="Student score")
args = parser.parse_args()

# Use args.name, args.score here
print("Name:", args.name)
print("Score:", args.score)
```

```
import argparse
parser =
argparse . ArgumentParser
(description = "Hospital
appointment tool")
parser . add - argument
("--name", type = str,
help = "Doctor name")
parser . add - argument
("time", type = str, help =
" Appointment time")
```

5. Adapt the skeleton to your project and note example commands at the bottom of your file.

python app.py -- name Ravi -- doctor Sharma -- time 10 AM

```
4) args = parser . parse - args()
print ("Name:", args . name)
print ("Doctor:", args . doctor)
print ("time:", args . time)
```

**Activity A8 – Mini Capstone: End-to-End Project Plan & Review**

Title: From Zero to Demo

Goal

Produce a realistic plan and self-review for your project.

**Part A – Milestones and Timeline**

1. List 4–6 milestones (each should be testable):
   - Milestone 1: Design input system
   - Milestone 2: Implement appointment storage
   - Milestone 3: Develop display functionality
   - Milestone 4: Add error handling
   - Milestone 5: Test the system
2. For each milestone, define:

| Milestone | Concrete Deliverable | Due Date |
|---|---|---|
| Input system, storage | User can enter details | Day 1 and 2 |
| Display, Errorhandling | Data storage in list / file | Day 3 and 4 |
| Testing | All features checked | Day 5 |

**Part B – Risk & Mitigation**

1. Identify 2–3 biggest risks (technical or time):
   - Risk 1: Logical errors in code
     - Plan B if this fails: Debug and test step by step
   - Risk 2: Not enough time
     - Plan B if this fails: Focus only on basic features
   - Risk 3: Data not saved properly
     - Plan B if this fails: Use simple file storage

**Part C – Final reflection after project**

(Do this at the end.)

1. When I got stuck, what strategies actually helped me get unstuck?

   I broke the problem into smaller parts and solved step by step
2. One thing I now believe about myself as a problem-solver:

   I can solve problems better by planning before coding
3. If I start a new project tomorrow, what will I do differently from day 1?

   I will clearly design the solution before starting coding.